



EDS PRACTICAL NO:-1

● NAME:-VEDANT DESHMUKH
PRN:-202501040046
BATCH :-C13
DIVISION:-CS1

PYTHON

1.1.1 CALCULATE MOMENTUM

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula:

$$p = m * v$$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

INPUT

```
m=float(input())  
v=float(input())  
p=m*v  
print(f"{p:.2f}kgm/s")
```



OUTPUT

Average time
0.007 s
7.25 ms

Maximum time
0.011 s
11.00 ms

✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 11 ms Debug

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s↵	50.00kgm/s↵

✓ Test case 2 6 ms Debug

Expected output	Actual output
2.3	2.3
4.8	4.8
11.04kgm/s↵	11.04kgm/s↵

1.1.2 CONDITIONAL CALCULATION BASED ON THE NUMBER OF DIGITS

Write a Python program that accepts an integer n as input. Depending on the number of digits in n .

Constraints:

$$1 \leq n \leq 999$$

Input Format:

The input consists of a single integer .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places). If n is a three-digit number, print its cube root (rounded to two decimal places). Else print "Invalid".

INPUT

```
n=int(input())
if 1<=n and n<=9:
    print(n**2)
elif 10<=n and n<=99:
    print(f"{n**(1/2):.2f}")
elif 100<=n and n<=999:
    print(f"{n**(1/3):.2f}")
else:
    print("Invalid")
```

OUTPUT

Average time 0.006 s 5.86 ms Maximum time 0.012 s 12.00 ms 4 out of 4 shown test case(s) passed 3 out of 3 hidden test case(s) passed

Test case 1 12 ms Debug	Expected output	Actual output
	9	9
	81	81
Test case 2 3 ms Debug	Expected output	Actual output
	166	166
	5.50	5.50
Test case 3 5 ms Debug	Expected output	Actual output
	24	24
	4.90	4.90
Test case 4 3 ms Debug	Expected output	Actual output
	3456	3456
	Invalid	Invalid

1.1.3 DAYS BETWEEN TWO DATES

Write a Python program to calculate the number of days between two dates.

Input Format:

- The first line contains the first date in the format : YYYY-MM-DD
- The second line contains the second date in the format :YYYY-MM-DD

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

INPUT

```
from datetime import date

#write your code here...
from datetime import datetime
#Read two dates as input
date1 = input().strip()
date2 = input().strip()

d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")

#calculate difference
difference= d2 - d1

#print number of days
print(difference.days)
```

OUTPUT

Average time
0.011 s
10.50 ms

Maximum time
0.017 s
17.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 17 ms Debug

Expected output

2014-07-02

2014-11-02

123↵

Actual output

2014-07-02

2014-11-02

123↵

✓ Test case 2 10 ms Debug

Expected output

2014-07-02

2014-07-11

9↵

Actual output

2014-07-02

2014-07-11

9↵

1.1.4 REVERSE A NUMBER

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

INPUT

```
num=int(input()) #Read the integer input

#Reverse the digits of the number
reversed_num = int(str(num)[::-1])

#Print the reversed number
print(reversed_num)
```



OUTPUT

Average time
0.007 s
7.00 ms

Maximum time
0.010 s
10.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 8 ms	Debug	⋮	📄	^
Expected output		Actual output		
5367		5367		
7635↵		7635↵		
✓ Test case 2 5 ms	Debug	⋮	📄	^
Expected output		Actual output		
0132		0132		
231↵		231↵		

1.1.5 MULTIPLICATION TABLE

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

INPUT

```
num=int(input())
for i in range(1,11):
    print(f"{num} x {i} = {num*i}")
```



OUTPUT

Average time: 0.009 s, Maximum time: 0.012 s, 2 out of 2 shown test case(s) passed, 2 out of 2 hidden test case(s) passed.

Test case 1 (12 ms) Debug

Expected output	Actual output
8	8
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40
8 x 6 = 48	8 x 6 = 48
8 x 7 = 56	8 x 7 = 56
8 x 8 = 64	8 x 8 = 64
8 x 9 = 72	8 x 9 = 72
8 x 10 = 80	8 x 10 = 80

Test case 2 (11 ms) Debug

Expected output	Actual output
5	5
5 x 1 = 5	5 x 1 = 5
5 x 2 = 10	5 x 2 = 10
5 x 3 = 15	5 x 3 = 15
5 x 4 = 20	5 x 4 = 20
5 x 5 = 25	5 x 5 = 25
5 x 6 = 30	5 x 6 = 30
5 x 7 = 35	5 x 7 = 35
5 x 8 = 40	5 x 8 = 40

LAB ASSIGNMENT

1.2.1 PASS OR FAIL

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

INPUT

```
def cal(marks,total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks=sum(marks)
    aggregate_percentage=(total_marks/(total_courses*100))*100
    if aggregate_percentage>75:
        grade="Distinction"
    elif aggregate_percentage>=60:
        grade="First Division"
    elif aggregate_percentage>=50:
        grade="Second Division"
    elif aggregate_percentage>=40:
        grade="Third Division"

    return(aggregate_percentage,grade)

num_courses=int(input())
marks=list(map(int,input().split()))

result =cal(marks,num_courses)

if result=="Fail":
    print("Fail")
else:
    aggregate_percentage,grade = result
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```



OUTPUT

Average time
0.013 s
13.00 ms

Maximum time
0.016 s
16.00 ms

✓ 5 out of 5 shown test case(s) passed
✓ 5 out of 5 hidden test case(s) passed

✓ Test case 1 13 ms Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75↵

Grade: First Division↵

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75↵

Grade: First Division↵

✓ Test case 2 13 ms Debug

Expected output

5

56 78 97 86 93

Aggregate Percentage: 82.00↵

Grade: Distinction

Actual output

5

56 78 97 86 93

Aggregate Percentage: 82.00↵

Grade: Distinction

✓ Test case 3 13 ms Debug

Expected output

5

99 85 43 97 34

Fail

Actual output

5

99 85 43 97 34

Fail

1.2.2 FIBONACCI SERIES

Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

INPUT

```
def fibonacci(n):  
    # write the code..  
    if n==1:  
        return 0  
    elif n==2:  
        return 1  
    else:  
        return fibonacci(n-1)+fibonacci(n-2)  
  
n = int(input())  
for i in range(1, n + 1):  
    print(fibonacci(i), end=" ")
```

OUTPUT

Average time
0.011 s
11.00 ms

Maximum time
0.022 s
22.00 ms

✓ 2 out of 2 shown test case(s) passed
✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 7 ms Debug

Expected output
5
0 1 1 2 3

Actual output
5
0 1 1 2 3

✓ Test case 2 7 ms Debug

Expected output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

Actual output
15
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377

1.2.3 PATTERN-1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

INPUT

```
n=int(input())
for i in range(1,n+1):
    print("*"*i)
```

OUTPUT

The screenshot displays the execution results of a Python program. At the top, performance metrics are shown: Average time 0.009 s (9.33 ms) and Maximum time 0.012 s (12.00 ms). Test results indicate that 2 out of 2 shown test cases passed and 4 out of 4 hidden test cases passed. Two test cases are detailed below:

Test Case	Expected output	Actual output
Test case 1	5 * ** *** **** *****	5 * ** *** **** *****
Test case 2	4 * ** *** ****	4 * ** *** ****

1.2.4 PATTERN-2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Note:

- Refer to the displayed test cases for the sample pattern.

INPUT

```
n=int(input())
for i in range(1,n+1):
    for j in range(1,i+1):
        print(j,end=" ")
    print()
```

OUTPUT

Average time: 0.011 s, 10.50 ms | Maximum time: 0.013 s, 13.00 ms | 2 out of 2 shown test case(s) passed | 2 out of 2 hidden test case(s) passed

Test case 1 (13 ms) [Debug]

Expected output	Actual output
5	5
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5

Test case 2 (9 ms) [Debug]

Expected output	Actual output
8	8
1	1
1 2	1 2
1 2 3	1 2 3
1 2 3 4	1 2 3 4
1 2 3 4 5	1 2 3 4 5
1 2 3 4 5 6	1 2 3 4 5 6
1 2 3 4 5 6 7	1 2 3 4 5 6 7
1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8